Lesson 13 Teacher's Key: A Hit That Reacts

Unreal Engine (optional advanced track) - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

Using a Blueprint Delay node to pause a Custom Event graph mid-execution, plus a Timeline-driven BP_Explosion Actor for the visual effect - both genuinely new this week, and a clean, idiomatic Blueprint way to express 'do this, pause, then do that.'

Level 1 - Full Worked Solution

- Delay(0.1) pauses only THIS execution wire within HandleEnemyDestroyed - every other Actor, widget, and graph in the level keeps running normally, the same non-blocking behavior as Godot's await and Unity's Coroutine.
- BP_Explosion is spawned and the enemy's Material swapped to M_Enemy_White BEFORE the Delay, so both happen immediately; EnemiesRemaining, the win-check Branch, and Destroy Actor are unchanged from Lesson 11/12, just running after the Delay instead of immediately.
- The flash is a real Set Material swap to a separate, pre-made Material (M_Enemy_White, built in this week's Setup from the supplied enemy_invader_white.png sprite) rather than a color tint - Set Vector Parameter Value("Color", White) would have been a documented no-op here, since M_Enemy_Red's Color parameter already defaults to White and multiplying by White again changes nothing. This is exactly the trap every other engine's own instructions warn students away from this week (see e.g. Unity Visual Scripting's and GameMaker's own trouble notes).
- BP_Explosion's own Timeline (BeginPlay -> scale up -> On Finished -> Destroy Actor) makes it fully self-contained, needing no help from the enemy or level Blueprint to clean itself up.

Why This Structure

- A Delay node is the right tool here specifically because it pauses in place within an existing Custom Event graph, without restructuring anything else - a much smaller change than GameMaker or TIC-80 need for the same idea, since Blueprint execution wires support pausing directly.
- Swapping to a separate white-sprite Material (rather than tinting one Material's color) matches every other engine's actual technique this week - Godot/Unity/GameMaker/GDevelop/TIC-80/001 Game Creator all swap in a genuine white version of the enemy image rather than trying to tint their way there, for the same

reason: a color multiply against an already-colored sprite can't produce a true, opaque white flash.

Level 2 / Level 3 Talking Points

- Ask students to trace the full execution path through HandleEnemyDestroyed now: sound, spawn explosion, flash Material, Delay, THEN everything from Lesson 11/12 unchanged - a good comprehension check for how a Delay node fits into an existing graph.
- For students comparing engines: Godot's await, Unity's Coroutine, and Unreal's Delay node all solve the identical problem with genuinely different underlying mechanisms, while GameMaker and TIC-80 need a structurally different approach (splitting into two events/checks) because their languages don't support pausing mid-script at all. The flash itself, meanwhile, is one of the few techniques that's genuinely identical across all eight engines this week: swap in a real white sprite/Material, don't try to tint one.

Common Student Mistakes

- Wiring Destroy Actor and the rest of the kill logic BEFORE the Delay node instead of after, which defeats the whole point (the enemy would still vanish instantly).
- Forgetting to set BP_Explosion's Timeline to auto-play, so it spawns but never actually animates or destroys itself.
- Reaching for Set Vector Parameter Value("Color", White) out of habit from Lesson 4/5, instead of Set Material(M_Enemy_White) - it compiles and runs with no error, but produces no visible flash at all, since White is already the Color parameter's neutral default.
- Editing M_Enemy_Red's own asset directly to try to change the look, instead of calling Set Material on the hit enemy - that would restyle every enemy in the level at once, not just the one that was hit.